

Le langage PASCAL permet au programmeur de définir, à partir des quatre *types standards* que nous avons vu précédemment, des types propres à un problème donné. De tels types sont dits *non standards* ; c'est le cas des *tableaux*, des *enregistrements*, des *fichiers*, des *chaînes de caractères*, des *énumérations*, des *intervalles*, des *ensembles*, et des *pointeurs*. Avant de les utiliser dans un programme PASCAL, il faut d'abord les déclarer. Les tableaux, les ensembles, les enregistrements et les fichiers constituent des *données structurées*.

Les *données structurées* sont des types de données composés de types élémentaires tels que integer, real, Char, type énuméré, type intervalle. Certains d'entre eux représentent des informations *homogènes* (type tableau (ARRAY)) quant aux autres, leurs données sont *hétérogènes* (le type enregistrement (RECORD), le type fichier (file) ... etc.).

15. LES ENUMERATIONS

Le type énumération est défini par une suite de valeurs ou d'identificateurs constituant un ensemble ordonné. Il représente une liste de constantes dont chacune, automatiquement, se repère par sa position au sein de cet ensemble. La première constante correspond à la position 0. Le type énuméré se déclare comme suit :

Type nom=(const1, const2,...) ; {Elles sont classées selon leur succession}

Exemple

a) Déclaration d'un type énumération identifié par jour :

TYPE jour = (dimanche, lundi, mardi, mercredi, jeudi, vendredi, samedi) ;

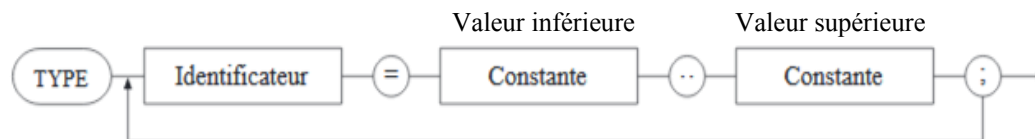
b) Déclaration d'un type énumération identifié par weekend :

TYPE weekend = (samedi, dimanche);

- ✓ Une fois définie ces variables peuvent être utilisées dans un programme grâce à des :
 - **Affectations** : $j_1 := \text{samedi}$; $j_3 := j_2$;
 - **Fonctions prédéfinies** :
 - **PRED** (lundi) donne dimanche ;
 - **SUCC** (jeudi) donne vendredi ;
 - **ORD** (dimanche) donne 0 ;
 - **Comparaisons** : dimanche < lundi < mercredi : suivant l'ordre donné dans la déclaration.
 - **Boucle**: for $C_1 := \text{lundi}$ to samedi do ;
- ✓ Il faut ajouter que, pour tout type énuméré, il existe la fonction réciproque (inverse) de **ORD**. C'est-à-dire une fonction qui, à un nombre entier, fait correspondre l'élément du type de rang correspondant. Elle porte simplement le nom du *type lui-même*. Ainsi, avec notre type jour : *jour (1)* a la valeur lundi.

16. LES INTERVALLES

Ce type décrit une suite de valeurs naturellement classées (nombres entiers ou caractères) limitée de part et d'autre par deux bornes constantes extrêmes. Sa déclaration est donnée par le diagramme syntaxique :



Ainsi, Le type intervalle, permet de définir un ensemble ordonné de valeurs ou d'identificateurs se situant entre une borne inférieure et une borne supérieure, toutes deux des constantes.

Exemple

```
Type mois=1 .. 12; {intervalle mois}
jour=1.. 31; {intervalle jour}
lettre_majus='A' .. 'Z' {intervalle alphabet majuscule}
lettre_minus='a'.. 'z' {intervalle alphabet minuscule}
chiffre=0..9 ;{intervalle chiffre}
Var m: mois;
    j: jour;
    Mlettre : lettre_majus;
    lettre : lettre_minus ;
    c: chiffre;
```

Déclaration d'un type intervalle identifié à partir d'un type énuméré :

```
TYPE jour = (dimanche, lundi, mardi, mercredi, jeudi, vendredi, samedi);
    Type jour_travail= dimanche..jeudi ;
    Type week_end=vendredi..samedi ;
```

17. LES ENSEMBLES

Dans le langage PASCAL, le type ensemble (SET) fait référence à un sous-ensemble d'un type de base préalablement défini et rentrant dans l'une des catégories suivantes : standard, énumération ou intervalle. La figure suivante présente le diagramme syntaxique de la déclaration d'un type ensemble, à partir d'un type de base préalablement défini.



Ainsi, Un ensemble est une collection d'éléments quelconque mais de types scalaire. Ces éléments ne sont pas manipulés individuellement mais en ensemble. Pour déclarer des variables de ce type, on procède de la manière suivante :

```
Type <nom de l'ensemble>= set of <type scalaire> ;
VAR VARIABLE : <nom de l'ensemble> ;
```

Ou bien :

```
VAR VARIABLE : set of <type scalaire> ;
```

Remarque

- Type scalaire ne doit pas comporter plus de 256 valeurs (exemple : CHAR, type intervalle, type énuméré)
- Les éléments d'un ensemble sont des valeurs ou identificateurs de l'ensemble de base généralement placés entre crochets. Un ensemble vide ne contient aucun élément à l'intérieur des crochets [].

Tout comme en mathématique, il est possible d'effectuer en PASCAL un certain nombre d'opérations de base sur les ensembles. Décrivons ces opérations.

- La **réunion** de deux ensembles (+) : la réunion de deux ensembles A et B est noté $A + B$ c'est l'ensemble de tous les éléments appartenant soit à A soit à B.
- L'**intersection** de deux ensembles (*) : leur intersection est notée $A * B$ et correspond à l'ensemble de tous les éléments appartenant à la fois à A et à B.
- La **différence** de deux ensembles (-) : la différence entre A et B est notée $A - B$ et correspond à l'ensemble des éléments présents dans A et absents dans B.
- L'**égalité** de deux ensembles (=) : l'égalité de deux ensembles A et B se note $A = B$ et exprime le fait que ces deux ensembles possèdent les mêmes éléments.
- L'**inégalité** de deux ensembles (<>) : l'inégalité ($\neq$) de deux ensembles A et B se note en PASCAL $A < > B$ et exprime le fait que ces deux ensembles ne contiennent pas exactement les mêmes éléments.
- L'**inclusion** d'un ensemble dans un autre ($=<$) : lorsque tous les éléments d'un ensemble A sont aussi éléments d'un ensemble B, on dit que A est inclus dans B, ce qui se traduit en PASCAL par la notation $A =< B$.
- L'**appartenance** à un ensemble (IN) : enfin, pour exprimer le fait qu'un élément a appartient à un ensemble A, en PASCAL, on écrit : $a \text{ IN } A$.

Exemple :

```
TYPE lettre = set of CHAR;
VAR OO,XX,ZZ,alphanum :lettre ;
Soit par exemple :
XX :=['A','C','E'] ;
YY:=['U','V','O'] ;
ZZ :=['A','Z','K'] ;
OO :=[ ] ; {ensemble vide}
Alphanum :=['A'..'Z','a'..'z','0'..'9'] ;
```

- $XX+YY=['A','C','E','U','V','O']$
- $XX*ZZ=['A']$,
- $XX*YY=[]$
- $XX*ZZ \subset XX+YY$ est true
- 'A' in XX est true
- $XX=YY$ est false,
- $XX<>YY$ est true

18. LES TABLEAUX

Jusqu'à présent, nous avons utilisé des variables de types simples qui peuvent mémoriser une seule valeur à la fois. Imaginons que dans un programme, nous avons besoin simultanément de 12 valeurs (*par exemple, des notes pour calculer une moyenne*). Evidemment, la seule solution dont nous disposons à l'état actuel de nos connaissances, consiste à déclarer douze variables, appelées par exemple : $N_1, \dots, N_{15}$. En arrivant au calcul, et après une succession de quinze instructions « **Lire** » distinctes, donnera obligatoirement une atrocité du genre :

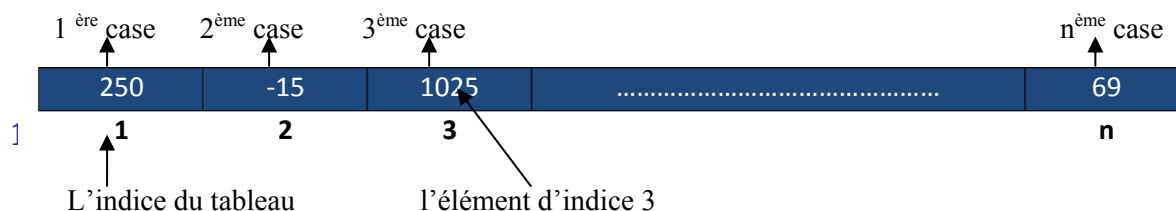
$Moyenne := (N_1+N_2+N_3+N_4+N_5+N_6+N_7+N_8+N_9+N_{10}+N_{11}+N_{12}+N_{13}+N_{14}+N_{15})/15 ;$

C'est pourquoi la programmation nous permet de rassembler toutes ces variables en une seule, au sein de laquelle chaque valeur sera désignée par un numéro. Pour le faire on doit utiliser des variables d'un type structuré qu'on appelle **Tableaux**. Il existe différents types de tableaux :

- ✓ Tableaux à une dimension (vecteur) ;
- ✓ Tableaux à deux dimensions (matrice) ;
- ✓ Tableaux à n dimensions.

18.1. Tableau à Une Dimension

Un tableau à une dimension ou un *vecteur* V d'ordre n (ou de cardinalité n) noté $V(n)$ est un ensemble de n *valeurs* de même *type* et portant le même *nom de variable*, et repérées par un nombre appelé *indice*, et qui sert à repérer chaque valeur du tableau séparément. A chaque fois qu'on doit désigner un élément du tableau, on fait figurer le nom du tableau, suivi de l'indice de l'élément entre deux crochets. Le schéma ci-dessous représente un tableau à une dimension.



Un tableau à une dimension est déclaré de la manière citée ci-dessous, en précisant le nombre et le type de valeurs qu'il contiendra.

Syntaxe

VAR

Nom_tableau : **Tableau** [valeur_indice_minimum Valeur_indice_maximum] **de** *type_des_éléments*;

Exemples :

Tab : tableau [1..100] d'entiers ;

Note : tableau [1..10] de réels ;

Remarques :

- ✓ Lorsqu'on déclare un tableau, on déclare aussi de façon implicite toutes les variables indicées qui le constituent.
- ✓ **En règle générale, l'indice minimum vaut 1.** Mais on peut aussi utiliser un autre indice minimum, comme 0. Dans ce cas, l'indice maximum sera égal au nombre d'éléments – 1.
- ✓ Lorsque l'indice minimum est 1, la **taille** du tableau (nombre d'éléments) est égale à la valeur de l'**indice maximum**.
- ✓ Certains langages acceptent que les valeurs minimales et maximales d'indice soient des variables. Mais comme ce n'est pas le cas de tous les langages, nous nous limiterons à des **valeurs d'indice constantes**.
- ✓ Le type de l'indice doit être un type ordinal c.à.d. entier, caractère mais jamais réel.
- ✓ Pour saisir ou afficher les éléments d'un tableau, on utilise les instructions d'écriture et de lecture dans des boucles (Pour, tant que, répéter).

18.1.2. Manipulation des Tableaux

Les éléments d'un tableau sont des variables indicées qui s'utilisent exactement comme n'importe quelles autres variables classiques. Autrement dit, elles peuvent faire l'objet d'une *affectation*, elles peuvent figurer dans une expression arithmétique, dans une comparaison, elles peuvent être *affichées* et *saisies*. On accède aux éléments d'un tableau en utilisant la syntaxe suivante :

Syntaxe

Nom_tableau [indice]

Exemples :

Si Tab est un tableau de 10 entiers alors :

- ✓ L'instruction **Tab[2] := 5** : permet de mettre la valeur 5 dans la deuxième case du tableau.
- ✓ En considérant le cas où 'A' est une variable de type entier ; l'instruction **A := Tab[2]** ; permet de mettre la valeur de la troisième case du tableau dans 'A' c.à.d. -5 ;
- ✓ **Lire Tab[1]** permet de mettre l'entier saisi par l'utilisateur dans la première case du tableau.
- ✓ **Afficher Tab[2]** permet d'afficher sur l'écran la valeur de la deuxième case du tableau.

Remarques :

- ✓ Il ne faut pas confondre l'indice d'un élément et la valeur de cet élément. En d'autres termes, il ne faut pas confondre i avec $\text{tab}[i]$;
- ✓ De même qu'une variable possède toujours une valeur (qui peut être indéterminée), un tableau peut très bien posséder des éléments dont la valeur est indéterminée. Donc avant d'utiliser un tableau, il est donc nécessaire de l'initialiser.

Exemple : Ecrire un algorithme permettant de remplir un tableau de 10 cases d'entiers, ensuite d'afficher le contenu des cases.

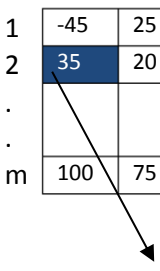
```
Algorithme Exemple ;  
CONST N=10 ;  
VAR  
  Tab: Tableau [1..N] d'entier ;  
  I : entier ;  
DEBUT  
  Pour i allant de 1 jusqu'à N faire  
    Lire (Tab[i]) ;  
  FinPour  
  Pour i allant de 1 jusqu'à N faire  
    Afficher (Tab[i]) ;  
  FinPour  
Fin.
```

18.2. Tableaux à Deux Dimensions

On appelle tableau à deux dimensions ou *matrice* A d'ordre $(m \times n)$ notée $A(m,n)$, un ensemble de $m \times n$ éléments rangés dans les cases disposés en m lignes et n colonnes. Dans ce type de tableaux les valeurs ne sont pas repérées par un seul indice comme les tableaux à une seule dimension, mais par deux indices (*coordonnées*), le premier indice sert à représenter les lignes et le second indice pour les colonnes. Le schéma ci-dessous représente un tableau à deux dimensions.

- ✓ Tous les éléments de la matrice doivent être de même type.
- ✓ Chaque élément de la matrice est repéré par le nom de la matrice et les numéros des indices de ligne et de colonnes, on le note $A[i, j]$.
- ✓ Lorsque $m=n$, alors la matrice est dite carrée
- ✓ Lorsque la matrice est carrée et si $i=j$, alors l'élément se trouve sur la diagonale principale de la matrice.

	1	2	3		n
1	-45	25	69		
2	35	20	155		
...					
m	100	75	1		



Ligne 2 colonne 1 = $A[2, 1] = 35$

18.2.1. Déclaration des Matrices

Un tableau à deux dimensions est déclaré de la manière citée ci-dessous, en précisant le nombre (*lignes et colonnes*) et le type des valeurs qu'il contiendra.

Syntaxe

VAR

Nom_matrice : **Tableau** [B_inf_ligne.... B_max_ligne, B_inf_colonne.... B_max_colonne] **de** *type_des_éléments*;

Exemple :

Achat : tableau [1..100, 1..100] d'entiers ;

Vente : tableau [1..10, 1..47] de réels ;

A : tableau [1..6, 'A'..'F'] d'entiers ;

Remarques

- ✓ A la déclaration d'une matrice, il faut ajouter la déclaration de deux indices qui doivent être d'un type ordinal.
- ✓ La saisie et l'affichage des éléments de la matrice sont effectuées par les instructions de lecture et d'écriture à l'intérieur de deux boucles imbriquées.

18.3. Déclaration par Référence

On peut aussi déclarer les tableaux d'une autre manière, en les considérant comme des types.

Exemple :

Type Tab : tableau [1..10] de réel ;

Mat : tableau [1..6, 1..5] d'entiers ;

VAR a, b : Tab ;

c, d : Mat ;